

Imperial Travel Agent

Project Guide and User Manual

MCP-first AI travel agent for campus routing, study-space planning, and weather-aware decisions

[Visit the site](#) [Visit the Repo](#)

Positioning

Imperial Travel Agent is a browser-based AI travel agent for Imperial College London students and visitors: after selecting a starting point, users can ask natural-language questions and receive study-space recommendations, live route plans, and weather-aware guidance.

Item	Details
Live website	https://ada-yz1825.github.io/Imperial_travel_agent/
Backend / fallback site	https://imperial-travel-agent-api.onrender.com/
Author	Yiqi Zhu
Programme	MSc Environmental Data Science and Machine Learning, Imperial College London
Document basis	Current uploaded code files and the public website interface checked on 25 May 2026

Note: Screenshots in this guide illustrate the current front-end layout and main interaction states. Live route times, weather data, and model/provider status may vary depending on API availability, backend cold starts, browser permissions, and the selected location.

1. Document Structure and Project Summary

This guide is written for two audiences: people who want to present the project clearly, and people who want to understand or maintain the code. The first half explains how the website works for users; the second half explains the MCP-first architecture, API routes, data sources, deployment model, and troubleshooting logic.

Section	What it covers
1	Document structure and project summary
2	Website feature tour
3	User operation guide
4	Weather, links, and supporting modules
5	Agent and MCP tool-calling logic
6	Technical architecture and runtime flow
7	Code file responsibilities
8	APIs, data sources, and environment configuration
9	Deployment, troubleshooting, and extension roadmap

Core value

The project merges study-place selection and travel planning into one natural-language entry point. The user does not need to decide whether they are asking for a study recommendation, navigation, weather, or general advice; the agent interprets the request and selects the right workflow.

FRONTEND	BACKEND	AGENT LAYER	EXTERNAL DATA
Static website	Python HTTP adapter	MCP tools	Google APIs + LLMs

2. Website Feature Tour

Overall layout from the hero area to the functional workspace

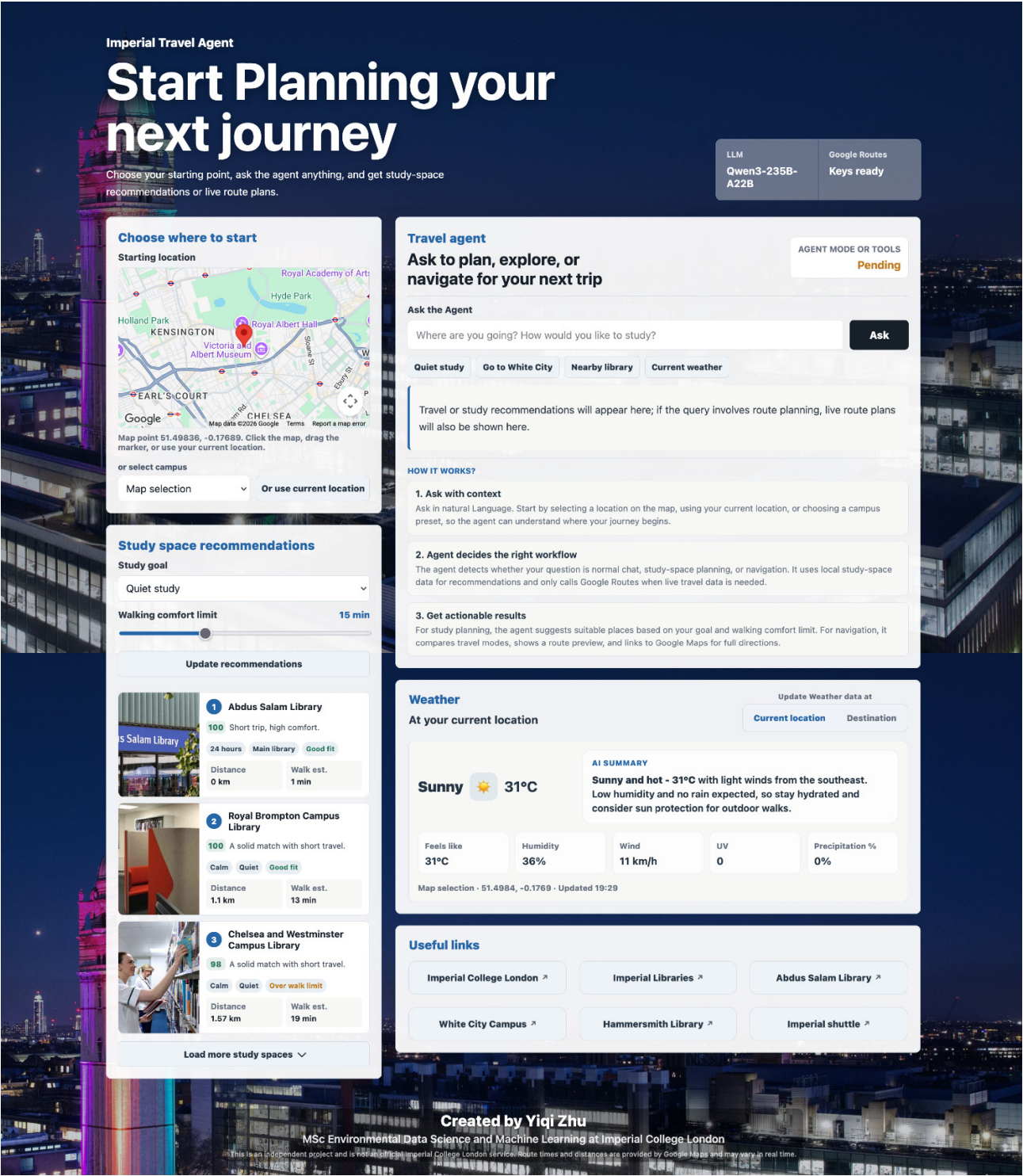


Figure 1. Home view and main workspace: start selection, study recommendations, agent panel, and status indicators.

The website uses a two-column workspace: a control panel on the left and an agent workspace on the right. The top hero area introduces the project and displays service status. The left column handles location and recommendation settings; the right column handles natural-language questions, route preview, weather, and useful links.

Module	Purpose
Status tiles	Show whether the LLM provider and Google Routes configuration are available; the startup modal also explains the possible backend warm-up delay.
Start selector	Lets the user select a point on Google Maps, use browser current location, or choose a campus preset such as South Kensington, White City, Hammersmith, or St Mary's.
Study recommendations	Ranks Imperial and public study spaces based on study goal, walking comfort, location, and available route estimates.
Travel agent	Natural-language entry point where the agent decides whether to answer directly or call MCP tools.
Route preview	Shows recommended travel mode, duration, distance, route map, mode switcher, and a Google Maps handoff link when the query involves navigation.
Weather module	Displays current conditions for the selected start point or navigation destination, including an AI-generated short summary.
Useful links	Provides quick access to official Imperial and campus-related pages, with an external-link confirmation step.

3. User Main Operation Guide

Typical workflow: choose a start point -> ask a question -> review route, weather, or recommendations

1. Study Space Recommendations

The first step is to define a starting point. This start point becomes the origin for route planning, the spatial context for nearby study-space recommendations, and the default location for weather queries.

Choose where to start

Starting location



Map point 51.49836, -0.17689. Click the map, drag the marker, or use your current location.

or select campus

Map selection ▼

Or use current location

Study space recommendations

Study goal

Quiet study ▼

Walking comfort limit 15 min

Update recommendations



1 Abdus Salam Library

100 Short trip, high comfort.

24 hours Main library Good fit

Distance 0 km

Walk est. 1 min



2 Royal Brompton Campus Library

100 A solid match with short travel.

Calm Quiet Good fit

Distance 1.1 km

Walk est. 13 min



3 Chelsea and Westminster Campus Library

98 A solid match with short travel.

Calm Quiet Over walk limit

Distance 1.57 km

Walk est. 19 min

Load more study spaces ▼

Figure 2-3. Left control panel: map or campus start selection, study goal, and walking-comfort threshold.

1. In “Choose where to start”, click the map, select a campus preset, or use “Or use current location” to let the browser provide a location.
2. In “Study space recommendations”, select a study goal such as Quiet study, Group study, Medical resources, Late study, or Nearest seat.
3. Adjust “Walking comfort limit” to control the acceptable travel or walking burden. A higher value allows more distant options to remain competitive.
4. Click “Update recommendations” to re-rank the cards. Study-space cards can be used as decision summaries and may also lead to relevant pages when a website is configured.

How to interpret the recommendation logic

The recommendation list is not simply sorted by straight-line distance. It combines study scenario, place tags, estimated travel effort, and walking comfort into a prototype score. When Google Routes is available, travel-time estimates can be refreshed with live API data; otherwise, the interface still provides a useful fallback ranking.

2. Travel Agent

After the user inputs the text to the Agent, the model begins to think. During this process, the model will automatically select the appropriate tools to provide the answer. The engineering process may take some time.

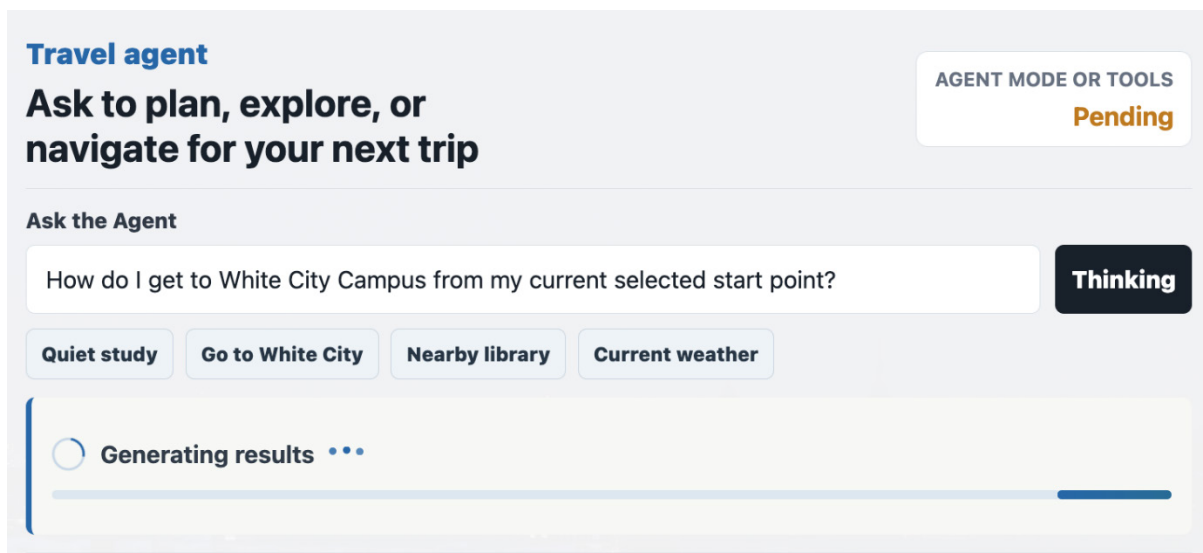


Figure 4. Loading interface before the agent outputs text.

When the user asks for directions or route comparison, the agent writes a short answer and opens the route preview below the answer area. The user can switch the transport mode to public transport, walking, cycling, or driving, then click Update to recalculate the displayed route.

Travel agent

Ask to plan, explore, or navigate for your next trip

AGENT MODE OR TOOLS

Navigation

Ask the Agent

How do I get to White City Campus from my current selected start point?

Ask

Quiet studyGo to White CityNearby libraryCurrent weather

Here's how to get to **White City Campus** from your current location:

Mode	Time	Distance	Note
Cycling	24 min	6.6 km	Fastest option via A402
Public transit	46 min	9.0 km	Direct routes available
Walking	76 min	5.5 km	Scenic but lengthy

Cycling is the quickest and most efficient choice. Let me know if you'd like real-time transit options or weather info for your trip! 🌤️

Route map: cycling24 min, 6.61 km

STARTMap selection on mapTRANSPORT MODE

DESTINATIONWhite City Campuscycling · 24 min, 6.61 kmUpdate

Open Google maps for RouteContinue in chat window

Expand introductions to the agent

Figure 5. Agent and route preview: natural-language request, recommended route, transport-mode switcher, and Google Maps handoff.

For follow-up questions, “Continue in chat window” opens a dedicated conversation modal. This is useful for asking why a place was recommended, adding constraints, or continuing from a previous route result.

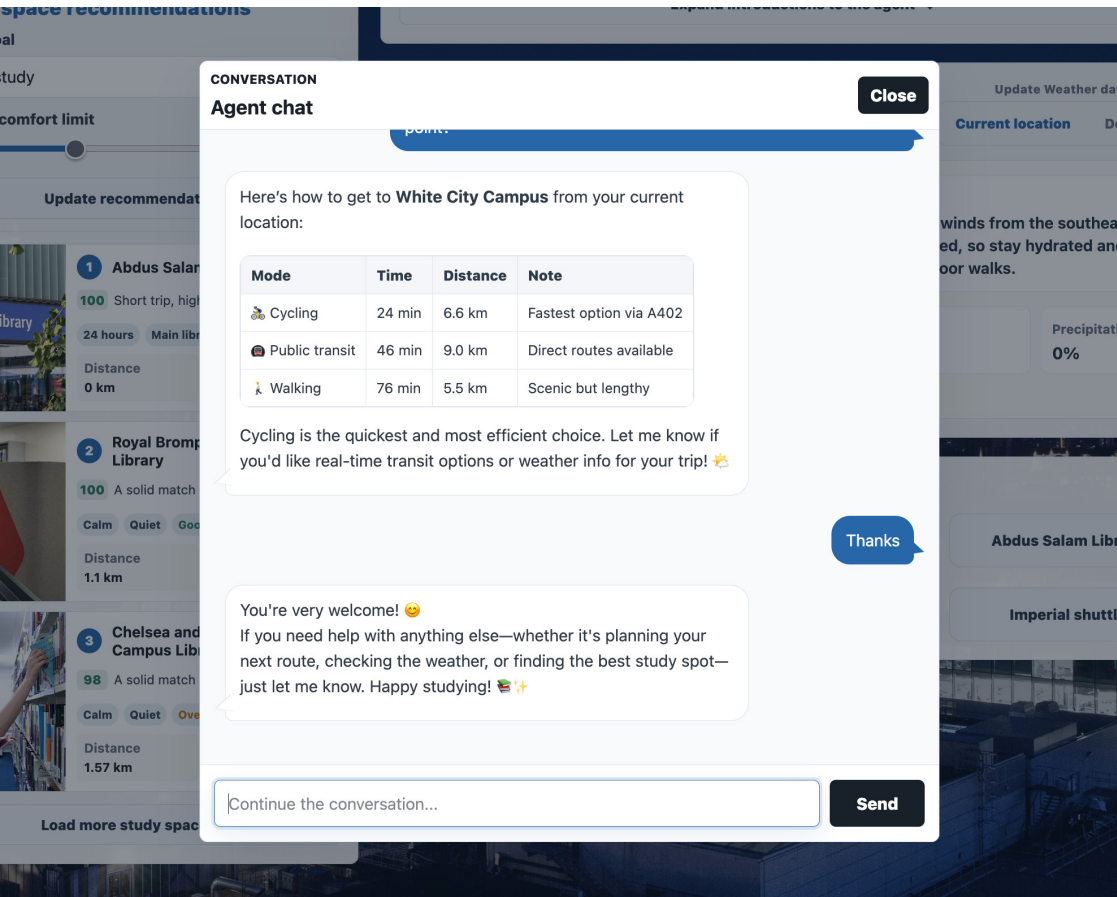


Figure 6. Chat modal: a focused space for follow-up questions and contextual conversation.

3. Weather Module

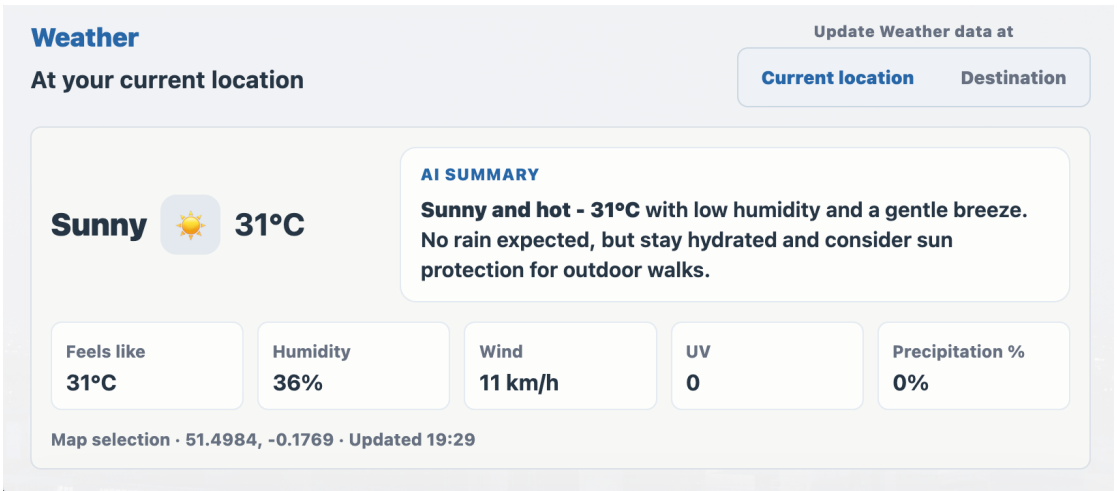


Figure 7. Weather module: Google Weather current conditions and a concise AI summary.

The Weather module supports better travel and study decisions by showing current conditions at the selected start point or at the latest navigation destination. The card includes temperature, feels-like temperature, humidity, wind, UV index, precipitation probability, and a short AI summary.

The Useful links module gives direct access to high-frequency official resources. When the user clicks an external link, the page shows a confirmation modal before opening a new tab, reducing accidental navigation away from the app.

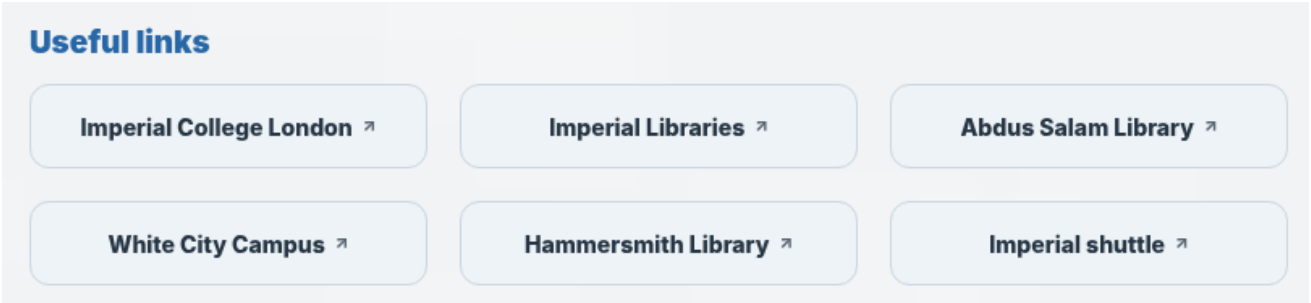


Figure 8. Useful links: Imperial website, library pages, campus pages, and shuttle-bus information.

Feature	How to use it	Output
Current-location weather	Click Current location in the Weather area	Shows real-time weather metrics for the selected or browser-derived start point
Destination weather	After route planning, click Destination	Shows weather for the navigation destination
External-link confirmation	Click Useful links or the Google Maps CTA	Confirms before opening a new browser tab
Chat continuation	Click Continue in chat window	Continues the conversation with recent context preserved

5. Agent and MCP Tool-calling Logic

The model chooses the workflow instead of following a fixed front-end route

The current version moves beyond a form-driven API pattern. The browser sends the question, selected start point, ranked study-space candidates, and recent chat history to the backend. The MCP layer then lets the tool-calling LLM decide whether to answer directly or call one or more tools such as navigation, route matrix, or weather.

User intent	Agent behavior
General chat or place information	Usually answered directly by the LLM without a route tool.
Study-space recommendation	Uses front-end ranked context and may call <code>route_matrix</code> when live travel estimates are useful.
Navigation or route comparison	Calls <code>navigate</code> to parse origin, destination, and mode, then requests Google Routes data.
Weather query	Calls <code>weather_current</code> using the selected start point, destination place, coordinates, or a resolvable place name.
System status check	Calls <code>health</code> to return LLM, Google key, MCP connection, and tool availability.

Why this counts as MCP-first

The browser no longer needs to own every capability decision. `server.py` acts mainly as the HTTP bridge; `mcp_server.py` exposes the tool registry and tool-calling loop; `navigator_core.py` executes the real LLM, Google Routes, Google Weather, geocoding, and parsing logic. This gives the project a clear MCP-first structure rather than a purely front-end-driven workflow.

MCP tool	Role
agent_answer	Main browser-chat entry point. Lets the tool-calling LLM decide whether to call other MCP tools.
chat_complete	Direct LLM-completion helper for ordinary answers and fallback paths.
classify_intent	Classifies whether the user question is closer to study planning, navigation, or normal conversation.
route_matrix	Requests Google Routes estimates from one start point to multiple study-space destinations.
navigate	Parses navigation requests, calls Google Routes, and returns recommended route, alternatives, and route geometry.
weather_current	Looks up current weather at a coordinate or named place.
health	Returns LLM, Google key, MCP, and tool status for the browser.

6. Technical Architecture and Runtime Flow

A thin front end, a layered backend, and an MCP tool layer for capability orchestration

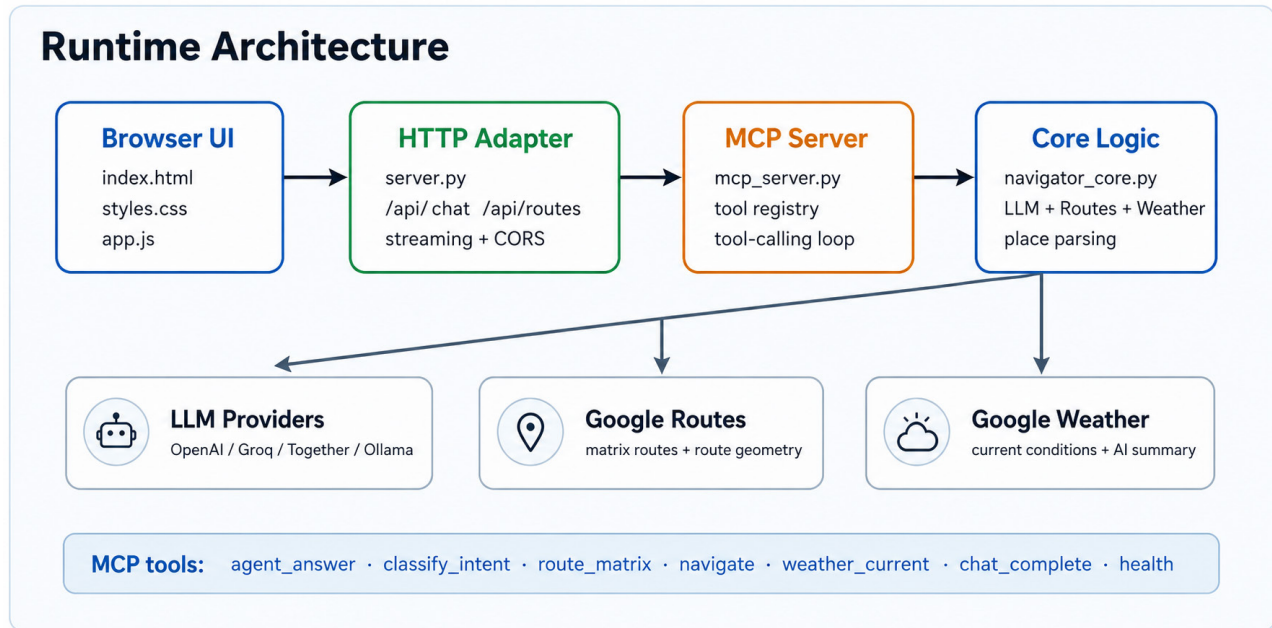


Figure 9. Technical architecture: Browser UI -> HTTP adapter -> MCP server -> core logic -> external services.

The runtime flow can be summarised in five steps:

1. `app.js` collects the user question, current start point, recommendation context, and chat history, then sends a request to `/api/chat`, `/api/navigate`, `/api/routes`, or `/api/intent`.
2. `server.py` works as the HTTP adapter. It handles CORS, JSON parsing, streaming responses, and communication with the MCP server through a stdio or HTTP MCP client.
3. `mcp_server.py` exposes `tools/list` and `tools/call` behavior, registers the available tools, and wraps tool outputs as `structuredContent`.
4. `navigator_core.py` performs the actual application logic: place parsing, geocoding, route requests, weather requests, LLM calls, answer cleaning, and local study-space data handling.
5. The front end renders structured results as answer text, tables, route preview maps, weather cards, chat bubbles, and action buttons.

Why this architecture is suitable for an agent project

The MCP layer separates “what the agent can do” from the visual interface. To add a new capability later, the main work should be extending the MCP tool schema, registry, and core execution function, while the front end only needs to understand the returned structured result.

7. Code File Responsibilities

A maintenance map for the current uploaded project files

File	Responsibility	Maintenance notes
index.html	Defines page structure: hero area, start selector, recommendation module, agent panel, route preview, weather, useful links, chat modal, and external-link confirmation modal.	Good place for copy, DOM structure, and panel-level changes. If an element id changes, update app.js accordingly.
styles.css	Defines the visual system, two-column layout, cards, buttons, animation, responsiveness, modal styles, route preview, weather card, and recommendation card styling.	Use for UI polish. Avoid unintended changes to key layout containers such as workspace, agent-module, and route-preview.
app.js	Handles front-end state, recommendation ranking, API calls, chat rendering, Markdown/table rendering, route map display, weather card display, external-link confirmation, and UI events.	Main front-end interaction file. New DOM elements or API fields will usually require changes here.
server.py	HTTP adapter exposing /api/health, /api/chat, /api/navigate, /api/routes, and /api/intent; manages CORS, streaming, and stdio/HTTP MCP client communication.	Keep the browser API stable. Prefer expanding capability through the MCP layer rather than overloading front-end endpoints.
mcp_server.py	MCP server and tool registry for agent_answer, chat_complete, classify_intent, route_matrix, navigate, weather_current, and health.	Central point for agent tool capability. New tools should be added to schema, registry, and handler logic.

navigator_core.py	Shared business core: LLM providers, Google Routes, Weather, Geocoding, place parsing, study-space data, route formatting, and answer cleaning.	Best place to maintain data, parsing rules, provider configuration, and external API integration.
README.md	Project overview, demo addresses, runtime flow, MCP tools, API endpoints, environment variables, and troubleshooting notes.	Update whenever architecture, deployment, environment variables, or public behavior changes.

The current deployment model is “static front end + runtime backend service”: GitHub Pages hosts the browser UI, while Render hosts the Python backend, MCP layer, LLM calls, and Google API calls.

```
Frontend: GitHub Pages - https://ada-yz1825.github.io/Imperial_travel_agent/  
Backend: Render - https://imperial-travel-agent-api.onrender.com/  
Runtime: browser -> server.py -> mcp_server.py -> navigator_core.py -> LLM / Google APIs
```

8. APIs, Data Sources, and Environment Configuration

server.py exposes a small and stable set of HTTP endpoints to the browser. The actual agent capabilities are implemented behind the MCP server, so the front end does not need to know the internal details of every tool.

Endpoint	Method	Purpose
/api/health	GET	Returns backend, LLM, Google key, and MCP tool status.
/api/chat	POST	Main browser-chat entry point. Supports streaming NDJSON or normal JSON responses.
/api/navigate	POST	Direct navigation entry point. Parses origin/destination and returns route JSON.
/api/routes	POST	Route-matrix entry point used by study-space recommendations.
/api/intent	POST	Classifies whether a user question is closer to study planning or navigation.

The project uses four main categories of data and services:

- Local study-space metadata: Imperial libraries, public libraries, and related learning locations are defined in navigator_core.py.
- Google Routes: provides live route-matrix estimates, single-route duration and distance, and polyline geometry for the route map.
- Google Weather: provides current weather conditions; the front end can pair these with a short LLM-generated summary.
- LLM providers: OpenAI, Groq, Together, and Ollama are supported for agent answers, intent classification, place extraction, and summary generation.

Environment variable	Usage
----------------------	-------

LLM_PROVIDER	Selects openai, groq, together, or ollama.
OPENAI_API_KEY / GROQ_API_KEY / TOGETHER_API_KEY	API keys for cloud LLM providers.
OPENAI_MODEL / GROQ_MODEL / TOGETHER_MODEL / OLLAMA_MODEL	Model name for the selected provider.
GOOGLE_MAPS_API_KEY	Backend key for Google Routes and Geocoding calls.
GOOGLE_MAPS_BROWSER_KEY	Browser-facing Google Maps / Weather key where applicable.
IMPERIAL_MCP_SERVER_URL	Optional remote MCP server endpoint for server.py.
IMPERIAL_MCP_COMMAND	Optional command for launching a local MCP server.
CORS_ALLOWED_ORIGINS	Restricts which front-end origins may call the backend.

```
export LLM_PROVIDER=together
export TOGETHER_API_KEY=your_together_key
export GOOGLE_MAPS_API_KEY=your_routes_key
export GOOGLE_MAPS_BROWSER_KEY=your_browser_maps_and_weather_key
python3 server.py
```

9. Deployment, Troubleshooting, and Extension Roadmap

Scenario	Check / fix
The page shows Checking or Pending after first load	The Render backend may be cold-starting. Wait briefly, refresh, or click the status tile for the built-in explanation.
The agent cannot answer	Check <code>/api/health</code> and confirm that <code>LLM_PROVIDER</code> and the matching API key are configured.
Google Routes is unavailable	Confirm <code>GOOGLE_MAPS_API_KEY</code> , Routes API enablement, billing, and key restrictions in Google Cloud.
Map or weather does not load	Check <code>GOOGLE_MAPS_BROWSER_KEY</code> , referrer restrictions, Maps JavaScript API, and Google Weather access.
GitHub Pages was updated but the website still looks old	Wait for Pages build/cache refresh; verify that the updated files were committed to the correct branch and path.
Local CORS issue during development	Check <code>CORS_ALLOWED_ORIGINS</code> or use one of the localhost ports already allowed by the server defaults.

The project is already suitable for coursework or portfolio demonstration, but the following limitations should be stated clearly when presenting it:

- It is an independent project, not an official Imperial College London service. The website footer already states this boundary.
- Route time and distance depend on Google Maps/Routes live responses and can change with traffic, service disruption, and API conditions.
- Crowd level, comfort, and study-space suitability are currently prototype estimates, not live occupancy data.
- External API keys, Render cold starts, and browser location permissions can affect the user experience.

Recommended next development direction

The next step is to move study-space metadata out of code and into a JSON/config registry, then add more live data such as opening hours or occupancy. Additional MCP tools could support campus facility lookup, accessibility-aware route filtering, library opening hours, and persistent user preferences.

Improvement	Expected value
Study-space JSON/config registry	Makes place data, tags, images, opening hours, and scoring fields easier to maintain.
Opening-hours or occupancy integration	Improves real-time recommendation reliability.
Tool-level logging and analytics	Makes it easier to debug why the agent did or did not call a specific tool.
Automated tests for parser and MCP tools	Reduces regression risk when changing route parsing, weather lookup, or provider configuration.
Accessibility polish	Improves keyboard navigation, screen-reader labels, and mobile experience.

Source basis: README.md, index.html, styles.css, app.js, server.py, mcp_server.py, navigator_core.py, and the public website interface at the time of document generation.